

Debian Package Installation

Install *nftfw* from the Debian Package

nftfw is installed from a Debian binary package, there is a zip file called *nftfw_current.zip* in the [package directory](#) containing the most recent version. For safety, *nftfw* needs some configuration after installation. This document provides a step-by-step guide for installation.

Following Debian practice, the system is installed in the root of the file system, so the control files will be in */etc/nftfw* with the library files in */var/lib/nftfw*.

Getting started

This section presents the bare bones of installing the *nftfw* package on a vanilla system. To cope with some special circumstances, links in this document jump to sets of instructions later in the text, with the expectation that the reader will jump there and then return to just after the jump.

Iptables check

First check that you can upgrade your system to run *nftables*:

```
$ sudo iptables -V
iptables v1.8.2 (nf_tables)
```

If the output doesn't say *nf_tables*, then you need to swap your *iptables* version. See [Switching iptables](#) below, then come back here when you've done that.

Download the package

Github links and behaviour has changed since these documents were first crafted. To download the zipfile, visit [the package directory README page](#) and click on *nftfw_current.zip* on the page, the file should download automatically. The *zip* file is used to hide the version number, running *unzip* on the file will yield the package. The filename of the package contains a version number and ends in *.deb*, for example *nftfw_1.0.0-1_all.deb*.

Older versions of the code can be found on the same *README* page, if your system doesn't support Python 3.9, then the last version that should run on it is *nftfw_0.9.20-1_all.deb*, Clicking on the package name should automatically download the file.

Install the package

```
$ sudo dpkg -i nftfw_XXXXX_all.deb
```

where XXXXX is the version number of the file you downloaded. *dpkg* doesn't install dependencies and may complain and stop. If this happens run:

```
$ sudo apt --fix-broken install
```

which will install the dependencies and then install *nftfw*.

The *dpkg -i* command is also used to update a previously installed package to a new version.

When installing *nftfw*, you will be asked if you want to change the ownership of the */etc/nftfw* directory to allow configuration by a non-root user. When *nftfw* writes files

under the directory it will take the ownership from the owner of */etc/nftfw*. Debian's *sdeb-conf* is used to remember this setting for later updates, and you can change ownership after installation using:

```
$ sudo dpkg-reconfigure nftfw
```

What is installed?

The package will install:

- the Python commands in */usr/bin*: *nftfw*, *nftfwls*, *nftfwedit*, *nftfwadm*, *nftnetchk* and *nftfwan.a*
- Control files in */etc/nftfw*, unless they exist. The *rule.d* directory will be updated. The firewall is populated to permit access to commonly used services.
- Basic directory structure in */usr/var/lib/nftfw*.
- Manual pages for the commands above, and section 5 manual pages for *nftfw_config* and *nftfw_files*.
- Documentation and examples in */usr/share/doc/nftfw*.
- A cron file in */etc/cron.d/nftfw*, this will need editing to make active.
- *systemd* path file to enable monitoring of the directories in */etc/nftfw*.

Many directories have *README* files explaining what is there and why.

Check *nftfw* is running

Check that it's running:

```
$ sudo nftfw -x -v load
nftfw[15264]: Loading data from /etc/nftfw
nftfw[15264]: Creating reference files in /var/lib/nftfw/test.d
nftfw[15264]: Testing new nftables installation
nftfw[15264]: Testing nft rulesets from nftfw_init.nft
nftfw[15264]: Determine required installation
nftfw[15264]: No install needed
```

The number in the log is the process id, so will be different for you.

On first installation

See [Taking precautions if you have a live firewall](#) if your system is running a live *iptables* or *nftables* firewall, and you want to keep that active until *nftfw* is live and configured.

If you are running *nftfw* on a Sympl or Symbiosis system then you might want to migrate your current firewall settings into *nftfw* - see [Migrating a Sympl or Symbiosis firewall](#) below. It's a good idea to do this now, before starting systems that run *nftfw* automatically.

Loading the rules

If you are moving from an *iptables* system, it's important to flush the *iptables* system, use

```
$ sudo iptables-nft -F
$ sudo ip6tables-nft -F
```

it is possible to have both live *iptables* and *nftables* on the system, and you may get confusing results. Alternatively, the kernel may complain when *nftfw* tries to load its tables.

Load the *nftfw* rules into the kernel:

```
$ sudo nftfw -f -v load
```

nftfw will tell you what it's done. If the kernel complains, ensure that the *iptables* rules are flushed, and run the *load* command again.

Look at the nftables rules

```
$ sudo nft list ruleset ip | less
```

for ipv4 and

```
$ sudo nft list ruleset ip6 | less
```

for ipv6. Hint: this is a lot to type and you may want to use the commands again, so create and store shell aliases in your shell's `.rc` file for them.

```
alias nfl='sudo nft list ruleset ip|less'
alias nfl6='sudo nft list ruleset ip6|less'
```

In extremis, you can clear the rules with

```
$ sudo nft flush ruleset
```

Changing config.ini

The `nftables.conf` file is the input configuration file for the `nftables` system and is the output of `nftfw`. For safety, the distributed version writes the file in `/etc/nftfw/nftables.conf`. The file here can be deleted. You need to tell `nftfw` to write the file in the correct place - in `/etc/nftables.conf`. With the `nftables.conf` file installed in `/etc`, the `nftables` system will load from it on system boot. Once the system is running, `nftfw` will reload the kernel from this file when a new version is created.

Edit `/etc/nftfw/config.ini` to correctly site the `nftables.conf` file:

```
# Location of system nftables.conf
# more comments...
# Usually /etc/nftables.conf
nftables_conf = /etc/nftables.conf
```

run `nftfw` to write the file, and also to load the kernel's `nftables`:

```
$ sudo nftfw -f -v load
```

Start the nftables service

Check that `nftables.service` is running:

```
$ sudo systemctl status nftables
```

and if not:

```
$ sudo systemctl enable nftables
$ sudo systemctl start nftables
```

Changing /etc/cron.d/nftfw

Edit the `/etc/cron.d/nftfw` file to make the working lines active, removing the `#` from the start of the lines containing cron commands.

Start the active control directories

```
$ sudo systemctl enable nftfw.path
$ sudo systemctl start nftfw.path
```

making `nftfw` run when anything changes in the `incoming.d`, `outgoing.d`, `blacklist.d`, `whitelist.d` and `blacknets.d` directories in `/etc`.

Running on SympI/Symbiosis?

SympI has a cron job to reload its firewall and this must be removed. Move `/etc/cron.d/sympI-firewall` to a safe place, so you can re-install it if you want to revert to the distributed firewall system.

Also for Syml, remove or move two links to `/usr/sbin/syml-firewall` under `/etc/network`:

```
$ cd /etc/network
# put into a safe place - in case you want to revert
$ sudo mv if-up.d/syml-firewall ~/up-syml-firewall
$ sudo mv if-down.d/syml-firewall ~/down-syml-firewall
```

Symbiosis has similar files prefixed by *symbiosis* that should be removed or saved.

You are done

If you are new to *nftfw*, look at the [How do I...](#) document which has sections on how to add or remove firewall controls. It should get you going on how to configure the firewall. As distributed, *nftfw* allows access to most of the usual services supplied by a LAMP system.

You now have an active *nftfw* system and should look in `/etc/nftfw` to configure the various control directories to your system needs.

More complex scenarios

This section contains extra command sequences and information, that are referenced above for special circumstances.

Switching iptables

Here is what to do if `iptables -V` says 'legacy' and not 'nf_tables':

```
$ sudo iptables-save > ipsaved
$ sudo ip6tables-save > ip6saved
$ sudo update-alternatives --config iptables
# select selection 0, /usr/sbin/iptables-nft, auto mode
$ sudo update-alternatives --config ip6tables
# select selection 0, /usr/sbin/ip6tables-nft, auto mode
```

Run the `sudo iptables -V` again, to check things have switched, and

```
$ sudo iptables-restore < ipsaved
$ sudo ip6tables-restore < ip6saved
$ sudo iptables-legacy -F
$ sudo ip6tables-legacy -F
```

The last two commands are very important to clear out the old tables. On Debian systems after Bullseye, there is better protection against having both *iptables* and *nftables* active in the same kernel. These commands are needed to flush tables, otherwise the `load` command to *nftfw* will fail.

```
$ sudo iptables-nft -F
$ sudo ip6tables-nft -F
```

[Back to Install the package](#)

Precautions for a live firewall

If have a running *nftables* or *iptables* firewall, then it's a good idea to save its rules in *nftfw*'s internal backup system so that the system will revert to your working firewall on a problem.

If you have a running firewall, save its rules first, and then load the *nftfw* rules:

```
$ sudo nftfwadm save
$ sudo nftfw -f -v load
```

Output should end with 'Install rules in ...' - wherever the `config.ini` file tells *nftfw* to store the `nftables.conf` file. The new rules will be installed in the kernel tables:

```
$ sudo nft list ruleset
```

will list the ruleset which will have been changed by *nftfw*.

If you have a problem, revert to old rules:

```
$ sudo nftfwadm restore
```

if not

```
$ sudo nftfwadm clean
```

What's happening here? The first *nftfwadm save* saves the current settings into *nftfw*'s backup file. In the event of *nftfw* failing, it will revert to the saved information. You can make this happen by using *restore*. When testing is over, it's also important to run the *clean* command, because *nftfw* won't create a safety backup file if one exists.

[Back to Loading the rules](#)

Migrating a Sympl or Symbiosis firewall

If you are installing *nftfw* on a Sympl or Symbiosis system then read this section.

The Debian package is supplied with a python script in */usr/share/doc/nftfw/import_tool*. It can import all the firewall settings from *incoming.d*, *outgoing.d*, *blacklist.d* and *whitelist.d* into *nftfw*. The script contains a lot of built-in information and sample commands. The script is also available in the *import_tool* directory in the *nftfw* source release.

```
$ cd /usr/share/doc/nftfw/import_tool
```

```
$ ./import_to_nftfw.py | less
```

will give you the basic information. Running the output through *less* will help with seeing the output. When run with action arguments, the script will tell you what it intends to do. Arguments are needed to force it to write files. The idea is look and check, then write files by adding an argument. You'll need to use *sudo* to update things.

Try:

```
$ ./import_to_nftfw --rules
```

to see what rules will be used by the new firewall files. The script understands about the *local.d* directory and will flag up any local scripts that will need porting into the *nftfw* system.

Once you've updated the firewall, run *nftfw* to load the new settings:

```
$ sudo nftfw -f -v load
```

you can check the rules using the *nft* commands

If are are here from the text above, [return to Loading the rules](#).